NEXALUD

S.I.G.S.M.

Programación FullStack

Nexalud

Rol	Apellido	Nombre	C.I	Email
Coordinador	Banfi	Ezequiel	5.722.220-9	ezebanfi2008@gmail.com
Sub-Coordinador	Leites	Thiago	5.748.207-9	thiagoleiteslt2021@gmail.com
Integrante 1	Almeida	Sabrina	5.779.656-3	sabrinaalmeida0509@gmail.com
Integrante 2	Gómez	Mathias	5.785.556-9	mathiasgomez517@gmail.com

Docente: Acosta, Fabián

Fecha de culminación

02/09/2026

SEGUNDA ENTREGA

Índice

1. Justificación tecnológica (PHP, MariaDB, Bootstrap, Git — relacionado a necesidades del sistema). 4

1.1 PHP 4

1.2 MariaDB 4

1.3 Bootstrap 5

1.3.1 Interfaces administrativas 5

1.3.2 Interfaces para usuarios (mobile-first) 5

1.3.3 Sistema de grillas responsivo 5

1.4 Git y Github 6

1.5 Visual Studio Code 6

1.6 Relación 6

2. Diseño 7

3. DER 7

3.1 Modelo Entidad-Relación (DER) 7

3.2 Restricciones No Estructurales 10

3.3 Esquema Relacional Normalizado 12

3.3.1 Pasaje del DER a tablas 12

3.3.2 Normalización 15

3.3.3 Registro de decisiones de diseño 15

3.4. Correcciones respecto a la primera entrega 16

3.5 Justificación del modelo 18

4. Manual de configuración 18

4.1 Requisitos de software con versiones específicas 18

4.2 Proceso de instalación paso a paso 19
4.2.1 XAMPP 19
4.2.2 Git 20
4.2.3 Visual Studio Code 20
4.3 Configuración de VS Code e implementación de extenciones 20
4.3.1 PHP Intelephense 22
4.3.2 Live Server 22
4.3.3 GitLens 23
4.3.4 Prettier 23
4.3.5 MYSQL 23
4.4 Instrucciones verificables 23
4.4.1 Verificación de XAMPP 23
4.4.2 Verificación de phpMyAdmin 24
4.4.3 Verificación de Git 24
4.4.4 Verificación de PHP 24
4.4.5 Verificación de Visual Studio Code 24
4.5 Lista de verificación final 24
5. Base de Datos **25**
5.1 Corrección del DER 25
5.1.2 Justificación 26
5.2 Implementación en PHPMyAdmin con scripts DDL 27
5.3 Evidencia de las tablas creadas 27
5.4 Datos de prueba 29
5.5 Restricciones no estructurales 30

1. Justificación tecnológica (PHP, MariaDB, Bootstrap, Git — relacionado a necesidades del sistema).

1.1 PHP

Utilizamos PHP como lenguaje de backend por su estabilidad, rendimiento y compatibilidad con servidores de GNU/Linux, lo que garantiza un soporte constante.

Otra ventaja importante es su integración nativa con MariaDB, haciendo así que sea mejor el manejo y almacenamiento de la información del sistema. Cuenta con una gran cantidad de documentación, y una gran comunidad de soporte que facilita el desarrollo y mantenimiento de la aplicación.

PHP nos permitió desarrollar módulos independientes que pueden integrarse con facilidad al sistema centralizado del hospital, asegurando flexibilidad y escalabilidad en el proyecto.

1.2 MariaDB

Implementamos MariaDB, porque nos permite guardar los datos clínicos y operativos de forma permanente, segura y confiable. Esta tecnología garantiza la integridad y consistencia de la información, evitando errores o pérdidas de datos en los registros de pacientes, ambulancias y traslados.

Además de eso, MariaDB es compatible con la infraestructura de Linux, y esto nos facilita su integración dentro del sistema institucional. Por otro lado, la combinación de PHP y MariaDB es uno de los entornos de desarrollo web más utilizados y documentados, facilitando el mantenimiento y capacidad de crecimiento en el proyecto.

1.3 Bootstrap

Utilizamos Bootstrap como framework frontend gracias a las ventajas que ofrece para el desarrollo de interfaces actuales, organizadas y adaptables a distintos tipos de dispositivos. Este framework nos permite agilizar el diseño de la página web mediante componentes ya diseñados y herramientas que mejoran la experiencia del usuario mediante el proceso de desarrollo.

1.3.1 Interfaces administrativas

Bootstrap nos proporciona componentes listos para usar, como tablas, formularios, botones, paneles y menús de navegación, que hace fácil el desarrollo de las interfaces administrativas. Esto permite organizar y gestionar información de manera clara, concreta y eficiente, reduciendo así el tiempo de programación y manteniendo, brindando una apariencia profesional y uniforme en todo el sistema.

1.3.2 Interfaces para usuarios (mobile-first)

Bootstrap está creado bajo una filosofía mobile-first, ¿Qué significa esto? Significa que las interfaces se desarrollan primero en dispositivos móviles y luego se adaptan automáticamente a dispositivos o pantallas más grandes. Esto es importante porque muchos usuarios acceden al sistema desde celulares o tablets, ya sea para consultar información, completar formularios o utilizar distintas funciones y opciones de la página web.

1.3.3 Sistema de grillas responsivo

Bootstrap incorpora un sistema responsivo que nos permite organizar automáticamente los elementos según el tamaño de la pantalla. Gracias a esto, la página puede verse correctamente en celulares, tablets y computadoras, garantizando el acceso, comodidad y una mejor experiencia de uso en cualquier dispositivo.

1.4 Git y Github

Para el desarrollo del proyecto utilizamos Git y GitHub como sistema de control de versiones, esto nos permite que varios integrantes del equipo trabajen en paralelo sin sobrescribir el código de los demás. Cada modificación realizada queda registrada con fecha, autor y descripción, lo que facilita el seguimiento y control de cambios durante el desarrollo.

Por otra parte, GitHub nos permitió mantener el proyecto organizado en un repositorio estructurado. El uso de Git y GitHub representa una buena práctica profesional, ya que es el estándar de control de versiones más utilizado en la industria del desarrollo de software.

1.5 Visual Studio Code

Se utiliza Visual Studio Code como entorno de desarrollo, por su facilidad de uso, rendimiento y compatibilidad con distintos tipos de tecnologías. Todo esto, nos permite añadir distintas extensiones que facilitan el trabajo en equipo y mejoran la productividad durante el desarrollo.

Una de las extensiones utilizadas es PHP Intelephense, que brinda autocompletado y detección de errores en PHP. Live Server, que permite visualizar cambios del frontend en tiempo real. GitLens, para consultar el historial de versiones añadidas en el editor. Prettier, se encarga del formateo automático del código para mantener consistencia entre los integrantes del equipo. Y la extensión Database Client (MySQL), que nos facilita la conexión y visualización de la base de datos directamente desde el IDE.

1.6 Relación

Las tecnologías seleccionadas responden directamente a las necesidades del sistema. Bootstrap, mediante su enfoque mobile-first, junto con PHP para servir brindar contenido

dinámico, permite que los usuarios accedan desde dispositivos móviles mediante códigos QR de forma cómoda y adaptable.

Para la gestión administrativa del hospital, se utiliza PHP junto con Bootstrap para desarrollar paneles de control organizados y funcionales en computadoras de escritorio. Por otro lado, MariaDB permite guardar los datos clínicos y operativos mediante una estructura organizada, segura y confiable

Finalmente, Git y GitHub facilitan el trabajo colaborativo del equipo, permitiendo mantener un repositorio organizado, controlar versiones y registrar los cambios realizados durante el desarrollo del proyecto.

2. Diseño

El diseño responsivo de ambos módulos se resuelve con el sistema de grillas de Bootstrap, que está construido sobre Flexbox. Se utilizan propiedades como flex-direction, justify-content y align-items para los ajustes puntuales que la grilla no cubre, lo que garantiza alineación y espaciado consistentes con menor complejidad de CSS.

La amplia compatibilidad de Flexbox asegura un acceso fiable desde distintos dispositivos y mejora la usabilidad tanto para el personal médico y administrativo (escritorio) como para los pacientes (móvil).

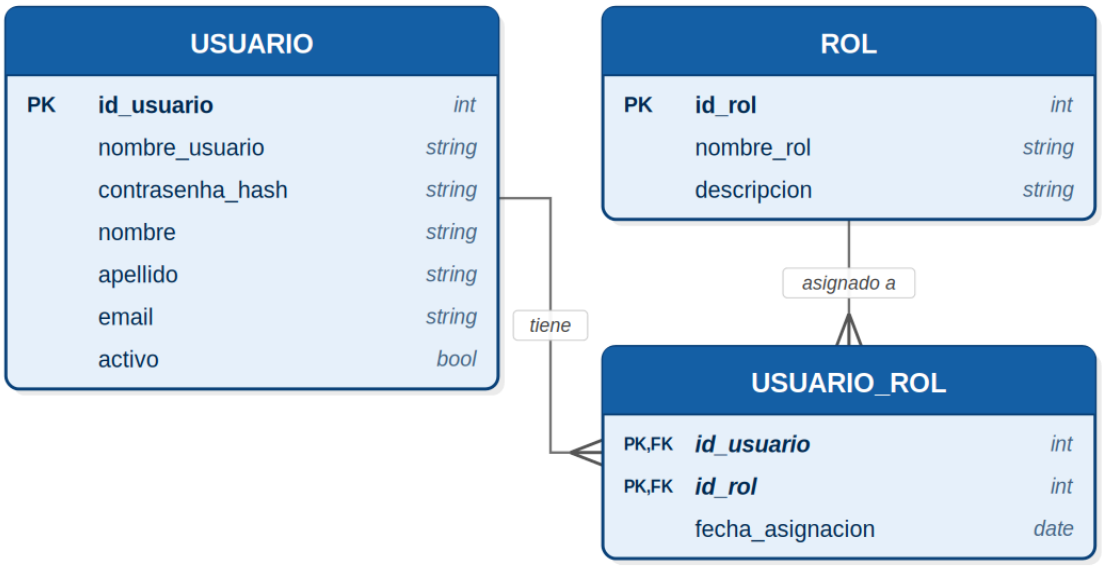
3. DER

3.1 Modelo Entidad-Relación (DER)

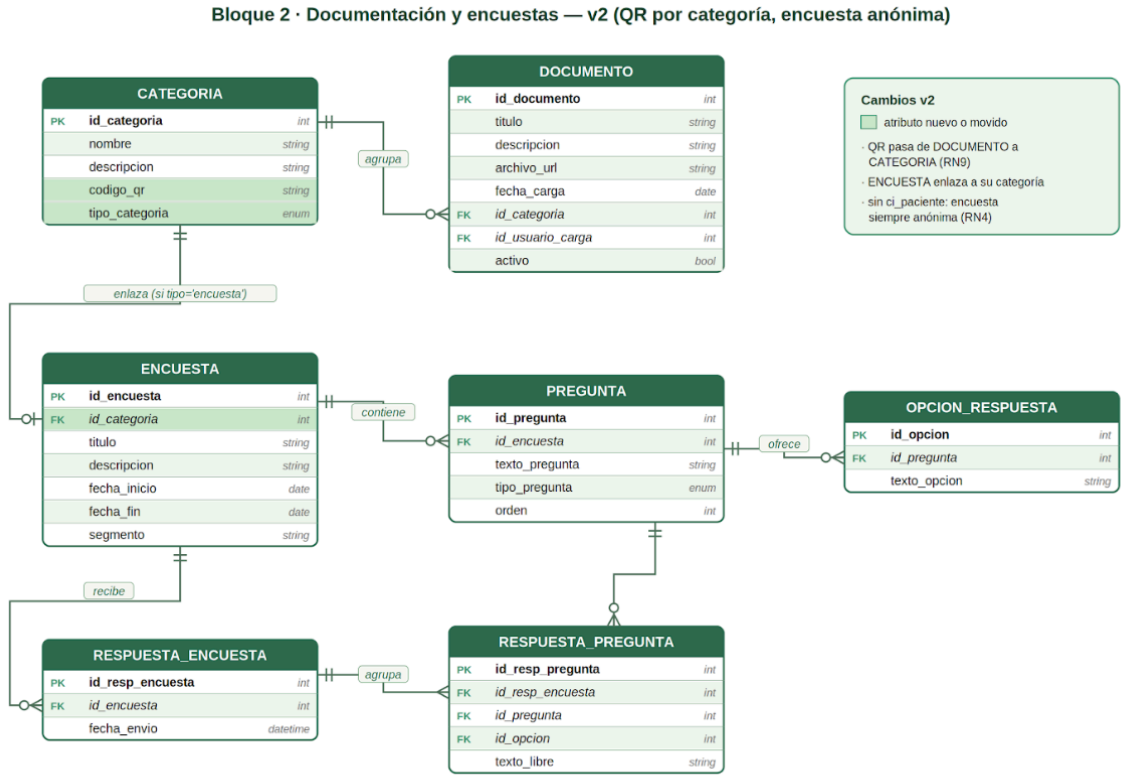
Para representar la estructura de datos del sistema del Hospital de Clínicas se elaboró un Modelo Entidad-Relación (DER). Dada la cantidad de entidades involucradas, el modelo

se dividió en tres bloques correspondientes a los módulos de usuarios y autenticación, gestión documental y encuestas, y trazabilidad de ambulancias. En ellos se identifican las entidades principales, sus atributos, claves primarias, claves foráneas y las relaciones existentes entre ellas.

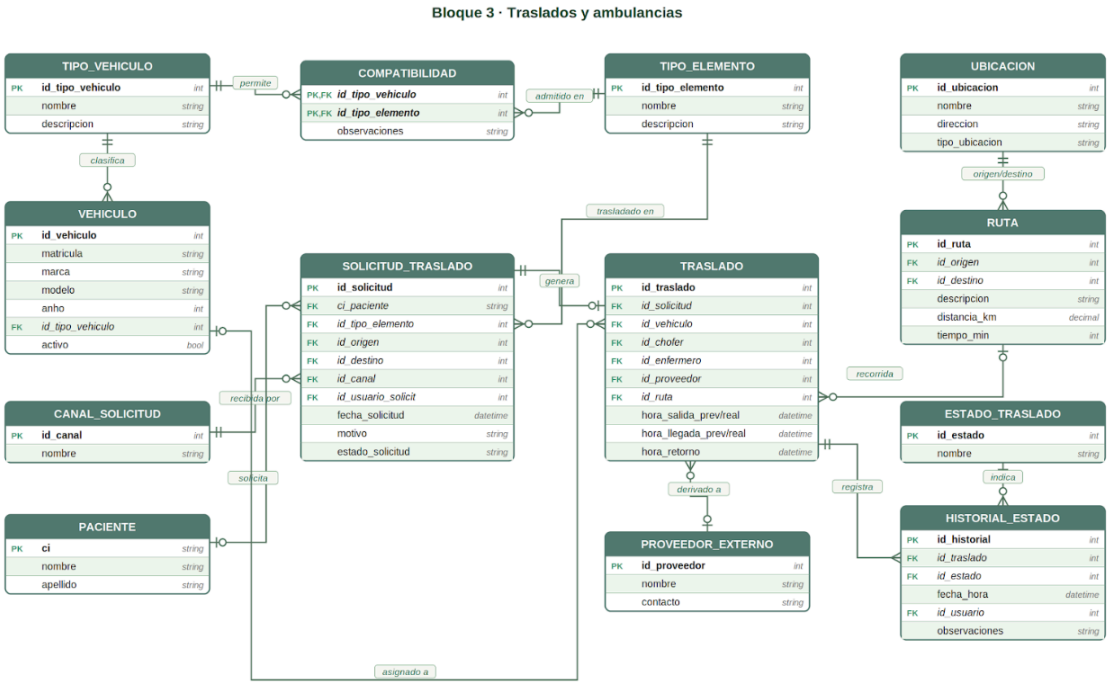
Bloque 1 – Usuarios y autenticación



Bloque 2 – Gestión documental y encuestas



Bloque 3 – Trazabilidad de ambulancias



3.2 Restricciones No Estructurales

Las restricciones no estructurales corresponden a reglas de negocio que el sistema debe cumplir, pero que no se pueden representar de forma directa en el Modelo Entidad-Relación. Estas reglas se implementan mediante validaciones en PHP, restricciones de la base de datos o una combinación de ambas.

Descripción de la regla	Entidades/Atributos involucrados	Implementación
Una solicitud de traslado puede ser para un paciente (cédula obligatoria) o para	Solicitud_Traslado, Paciente, Tipo_Elemento	Validación en PHP antes de guardar la solicitud.

un elemento no biológico (cédula nula y tipo de elemento distinto de paciente).		
Un traslado debe tener asignado un vehículo, un chofer y un enfermero del hospital, o bien ser derivado a un proveedor externo (SAME), pero nunca ambas opciones al mismo tiempo.	Traslado, Vehículo, Usuario, Proveedor_Externo	Validación en PHP y restricción en la base de datos mediante CHECK.
El vehículo asignado debe ser compatible con el tipo de elemento que se trasladará.	Vehículo, Tipo_Vehículo, Tipo_Elemento, Compatibilidad	Validación en PHP consultando la tabla Compatibilidad antes de asignar el vehículo.
Las respuestas de las encuestas anónimas no deben almacenar la cédula del paciente.	Encuesta, Respuesta_Encuesta	Validación en PHP guardando el campo de cédula como NULL cuando la encuesta sea anónima.
Los datos personales del paciente no podrán modificarse desde este sistema, ya que se obtienen mediante una API externa utilizando la cédula de identidad.	Paciente	La información se consulta desde la API y el sistema solo permite su lectura.

Un usuario activo debe tener al menos un rol asignado para poder utilizar el sistema.	Usuario, Rol, Usuario_Rol	Validación en PHP al crear o modificar usuarios y mediante claves foráneas en la base de datos.
El historial de estados de un traslado es inmutable; únicamente se podrán agregar nuevos registros y no modificar los existentes.	Historial_Estado, Traslado	Restricción mediante la lógica de la aplicación (PHP) y permisos sobre la base de datos.
Las rutas predefinidas funcionan como sugerencia; un traslado puede realizarse utilizando una ruta diferente cuando sea necesario.	Ruta, Traslado	Se permite que el campo correspondiente a la ruta sea opcional (NULL) y se valida desde PHP.

3.3 Esquema Relacional Normalizado

3.3.1 Pasaje del DER a tablas

Bloque 1 – Usuarios y autenticación

Rol(id_rol, nombre_rol, descripcion)

Usuario(id_usuario, nombre_usuario, contrasenha_hash, nombre,
apellido, email, activo)

UsuarioRol(id_usuario->Usuario, id_rol->Rol, fecha_asignacion) [PK compuesta]

Bloque 2 – Gestión documental y encuestas

Categoria(id_categoria, nombre, descripcion, codigo_qr [UNIQUE],
tipo_categoria {'documental' | 'encuesta'})

Documento(id_documento, titulo, descripcion, archivo_url, fecha_carga,
id_categoria->Categoria, id_usuario_carga->Usuario, activo)

Encuesta(id_encuesta, titulo, descripcion, id_categoria->Categoria [NULL],
fecha_inicio, fecha_fin, segmento)

Pregunta(id_pregunta, id_encuesta->Encuesta, texto_pregunta,
tipo_pregunta, orden)

OpcionRespuesta(id_opcion, id_pregunta->Pregunta, texto_opcion)

RespuestaEncuesta(id_resp_encuesta, id_encuesta->Encuesta, fecha_envio)

RespuestaPregunta(id_resp_pregunta, id_resp_encuesta->RespuestaEncuesta,
id_pregunta->Pregunta, id_opcion->OpcionRespuesta [NULL],
texto_libre [NULL])

Bloque 3 – Trazabilidad de ambulancias

Paciente(ci, nombre, apellido, fecha_nacimiento)

TipoVehiculo(id_tipo_vehiculo, nombre, descripcion)

Vehiculo(id_vehiculo, matricula, marca, modelo, anho,
id_tipo_vehiculo->TipoVehiculo, activo)

TipoElemento(id_tipo_elemento, nombre, descripcion)

Compatibilidad(id_tipo_vehiculo->TipoVehiculo,
id_tipo_elemento->TipoElemento, observaciones) [PK compuesta]

Ubicacion(id_ubicacion, nombre, direccion, tipo_ubicacion)

Ruta(id_ruta, id_origen->Ubicacion, id_destino->Ubicacion,
descripcion, distancia_km, tiempo_min)

CanalSolicitud(id_canal, nombre)

ProveedorExterno(id_proveedor, nombre, contacto)

EstadoTraslado(id_estado, nombre)

SolicitudTraslado(id_solicitud, ci_paciente->Paciente [NULL],
id_tipo_elemento->TipoElemento, id_origen->Ubicacion,
id_destino->Ubicacion, id_canal->CanalSolicitud,
id_usuario_solicit->Usuario, fecha_solicitud,
motivo, estado_solicitud)

Traslado(id_traslado, id_solicitud->SolicitudTraslado,
id_vehiculo->Vehiculo [NULL], id_chofer->Usuario [NULL],
id_enfermero->Usuario [NULL], id_proveedor->ProveedorExterno [NULL],
id_ruta->Ruta [NULL], hora_salida_prev, hora_salida_real,
hora_llegada_prev, hora_llegada_real, hora_retorno)

HistorialEstado(id_historial, id_traslado->Traslado,

id_estado->EstadoTraslado, fecha_hora,
id_usuario->Usuario, observaciones)

3.3.2 Normalización

Situación detectada	Forma violada	Corrección aplicada
Documento con lista de categorías embebida	1FN	CATEGORIA aparte; id_categoria como FK
Encuesta con preguntas embebidas	1FN	PREGUNTA y OPCION_RESPUESTA separadas
Respuestas en columnas (resp1, resp2, resp3…)	1FN	RESPUESTA_PREGUNTA: una fila por respuesta
Traslado con matrícula, marca y modelo del vehículo	3FN	Datos movidos a VEHICULO; se referencia por FK
Traslado con el nombre del chofer	3FN	Referencia por FK a USUARIO
Solicitud con nombre y dirección del origen	3FN	Referencia por FK a UBICACION

Todas las tablas del esquema final quedan en Tercera Forma Normal. No hay grupos repetidos, no hay dependencias parciales sobre claves compuestas, y no hay dependencias transitivas entre atributos no clave.

3.3.3 Registro de decisiones de diseño

Decisión adoptada	Alternativa descartada	Por qué
USUARIO único con ROL N:N	Entidades separadas por tipo de persona	Evita duplicar atributos; permite múltiples roles; un solo login
Separar SOLICITUD de TRASLADO	Una sola entidad	Permite rechazar o derivar sin ejecutar; respeta 3FN
HISTORIAL_ESTADO como tabla	Atributo estado en TRASLADO	El cliente pide trazabilidad de todo el ciclo, no solo el último estado
UBICACIÓN única para origen/destino	Dos entidades separadas	Mismo concepto con roles distintos; práctica estándar
COMPATIBILIDAD como tabla puente	CHECK fijo en código	La regla puede cambiar; mejor en datos que en código
Paciente con cédula como PK	Id sustituto	La cédula es la clave de integración con la API externa
Cabecera + detalle de respuestas	Una sola tabla de respuestas	Soporta número variable de preguntas; respeta 1FN

3.4. Correcciones respecto a la primera entrega

En la comparación entre el DER de la primera entrega y el DER corregido para esta segunda entrega, los cambios se concentran en las tablas de documentación y encuestas. Los bloques de autenticación y de traslados no presentaron modificaciones.

Entidad	Primera versión	Versión corregida	Por qué se corrigió

documento	Tenía atributo codigo_qr propio	Se elimina codigo_qr	El código QR deja de generarse por cada documento individual.
categoria	Solo tenía id_categoria, nombre, descripcion	Se agregan codigo_qr (único, generado automáticamente al dar de alta la categoría) y tipo_categoria ('documental' / 'encuesta')	La categoría pasa a ser el punto de acceso físico del paciente: cada categoría tiene su propio QR, y el tipo determina si ese QR muestra un listado de documentos o abre un formulario de encuesta.
encuesta	Entidad independiente, sin FK a categoría; tenía el atributo es_anonima (booleano)	Se agrega id_categoria (FK, nullable) y se elimina es_anonima	La encuesta debe poder resolverse desde el QR de una categoría de tipo 'encuesta'. El anonimato deja de ser una opción configurable, por lo que el campo booleano ya no tiene sentido.

respuesta_encuesta	Tenía ci_paciente como FK a paciente	Se elimina ci_paciente y su FK	La encuesta es siempre anónima: conservar la cédula, aunque fuera opcional, contradiría ese requisito y permitiría reconstruir quién respondió.
paciente	Participaba del Bloque 2, relacionado con respuesta_encuesta mediante la relación "completa"	Deja de participar del Bloque 2; solo se relaciona dentro del Bloque 3 (con solicitud_traslado)	Al eliminarse ci_paciente de respuesta_encuesta, paciente pierde todo vínculo con documentación y encuestas.

3.5 Justificación del modelo

usuario representa a cualquier persona que usa el sistema. Se eligió id_usuario como PK en lugar del nombre de usuario, porque este último podría cambiarse y no conviene usarlo como clave de relación. El atributo contrasenha_hash nunca guarda la contraseña en texto plano, y activo permite el borrado lógico sin romper referencias históricas.

rol se modela como entidad separada y no como atributo de usuario, porque un mismo usuario puede tener más de un rol a la vez. usuario_rol es la tabla puente que resuelve esa relación N:N, con clave primaria compuesta (id_usuario, id_rol).

categoría agrupa documentos y concentra el punto de acceso por código QR: cada categoría tiene su propio codigo_qr, generado al momento del alta. El atributo tipo_categoria ('documental' o 'encuesta') determina si el QR muestra un listado de documentos o redirige a un formulario.

documento ya no tiene codigo_qr propio, porque el acceso ahora ocurre a nivel de categoría.

encuesta, pregunta, opcion_respuesta, respuesta_encuesta y respuesta_pregunta forman el patrón cabecera-detalle que permite guardar un número variable de preguntas sin violar la 1FN. Las encuestas son siempre anónimas: ninguna de estas tablas guarda una referencia al paciente que respondió.

4. Manual de configuración

4.1 Requisitos de software con versiones específicas

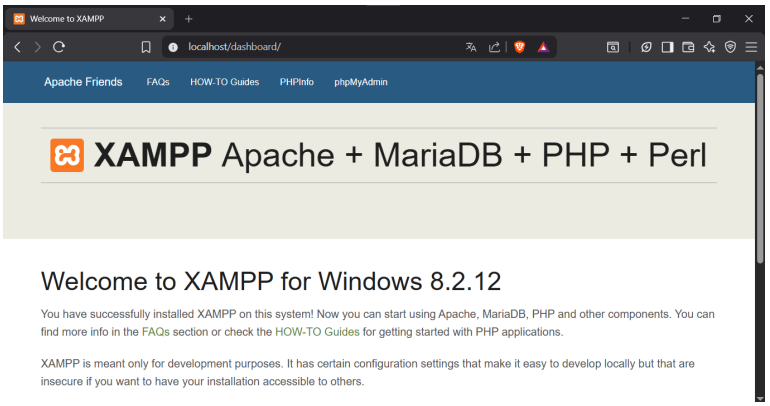
Herramienta	Versión específica	Descripción
XAMPP	8.2.12 (o superior)	Incluye Apache (servidor web), PHP y MySQL/MariaDB en un único instalador.
PHP	8.2.x (incluido en XAMPP 8.x)	Lenguaje de programación del backend.
MariaDB	10.4.x (incluido en XAMPP 8.x)	Sistema de gestión de bases de datos.
VirtualBox	7.0.x (o superior)	Para virtualizar el servidor de pruebas en GNU/Linux.
Git	2.44.x (o superior)	Para control de versiones local.
Visual Studio Code	1.89.x (o superior)	Editor de código principal.
Navegador web	Última versión	Para pruebas de interfaz.

4.2 Proceso de instalación paso a paso

A continuación se detalla el paso a realizar para la instalación de cada una de las herramientas mencionadas anteriormente.

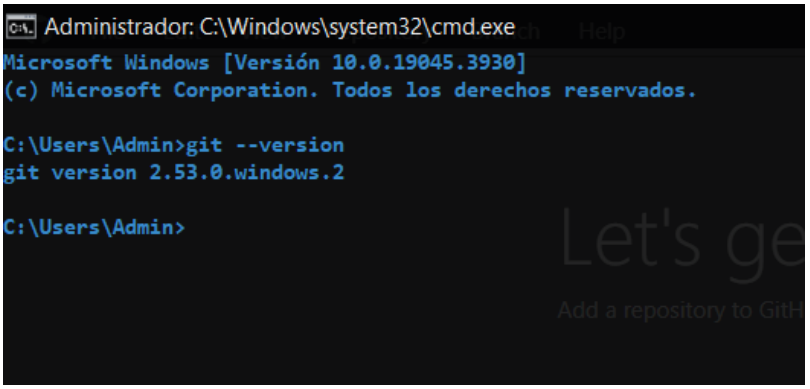
4.2.1 XAMPP

1. Ir a la descarga desde este link: https://www.apachefriends.org/es/download.html
2. Seleccionar la versión dependiendo del sistema operativo que se utilice y hacer clic en "descargar".
3. Ejecutar como administrador desde la carpeta descarga y seguir los pasos que diga.
4. Abrir el Panel de Control de XAMPP y hacer clic en "Start" en Apache y MySQL.
5. Verificar en el navegador:



4.2.2 Git

1. Ir a la descarga desde este link: https://git-scm.com/install/windows
2. Ejecutar como administrador y seguir los pasos por defecto.
3. Verificar en la terminal (CMD).



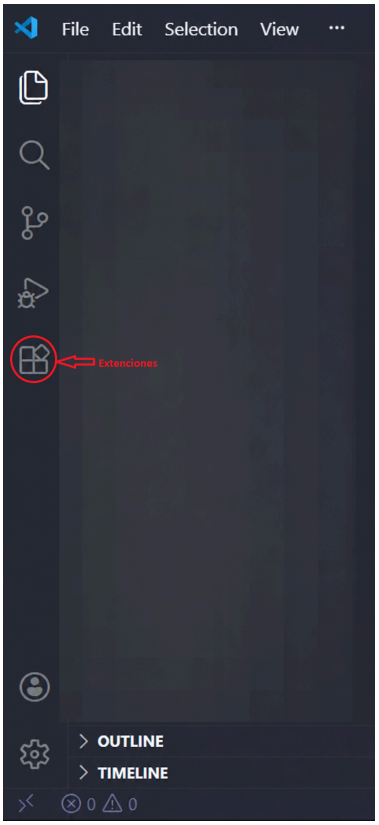
4.2.3 Visual Studio Code

1. Ir a la descarga desde este link: https://code.visualstudio.com/download
2. Ejecutar el instalador y seguir los pasos.
3. Abrir VS Code y verificar que inicia correctamente.

4.3 Configuración de VS Code e implementación de extenciones

Extensión	Descripción
PHP Intelephense	Proporciona autocompletado, detección de errores y ayuda para programar en PHP.
Live Server	Te permite ver la página web en tiempo real cada vez que le hagas una modificación nueva.
GitLens	Muestra el historial de commits, autores y cambios realizados en el proyecto.
Prettier	Te implementa color a todo tu código de manera que cada cosa esté organizada.

MySQL	Permite conectarse y administrar bases de datos desde Visual Studio Code.



4.3.1 PHP Intelephense

1. Abrir la pestaña Extensiones en VS Code.
2. Buscar "PHP Intelephense".
3. Instalar la extensión.
4. Reiniciar VS Code si es necesario.

4.3.2 Live Server

1. Abrir Extensiones.
2. Buscar "Live Server".
3. Instalar la extensión.
4. Hacer clic derecho sobre un archivo HTML y seleccionar “**Open with Live Server”**.

4.3.3 GitLens

1. Abrir Extensiones.
2. Buscar "GitLens".
3. Instalar la extensión.
4. Abrir un proyecto con Git para acceder a sus herramientas.

4.3.4 Prettier

1. Abrir Extensiones.
2. Buscar "Prettier - Code Formatter".
3. Instalar la extensión.

4.3.5 MYSQL

1. Abrir Extensiones.
2. Buscar "Database Client" o "MySQL".
3. Instalar la extensión.
4. Crear una nueva conexión utilizando:
    - Host: localhost
    - Puerto: 3306
    - Usuario: root
    - Contraseña: (según configuración del servidor)

4.4 Instrucciones verificables

Una vez que finalizó la instalación y configuración de todas las herramientas, se tiene que realizar las siguientes verificaciones para confirmar que el entorno de desarrollo funciona correctamente:

4.4.1 Verificación de XAMPP

1. Abrir un navegador web.
2. Ingresar el link: http://localhost

4.4.2 Verificación de phpMyAdmin

1. Abrir un navegador web.
2. Ingresar al link: http://localhost/phpmyadmin

4.4.3 Verificación de Git

1. Abrir la terminal o símbolo del sistema (CMD).
2. Ejecutar el comando: git --version

4.4.4 Verificación de PHP

1. Abrir la terminal o símbolo del sistema (CMD).
2. Ejecutar el comando: php -v

4.4.5 Verificación de Visual Studio Code

1. Abrir Visual Studio Code.
2. Crear un archivo con extensión .php.
3. Escribir código PHP básico.

4.5 Lista de verificación final

- XAMPP instalado correctamente.

- Apache iniciado correctamente.
- MySQL iniciado correctamente.
- La página http://localhost funciona.
- phpMyAdmin abre correctamente.
- Git responde al comando git --version.
- PHP responde al comando php -v.
- Visual Studio Code funciona correctamente.
- PHP Intelephense proporciona autocompletado.
- Todas las extensiones requeridas fueron instaladas correctamente.

5. Base de Datos

5.1 Corrección del DER

En la comparación realizada con el DER de la primera entrega y el DER corregido para esta segunda entrega, los cambios se centran específicamente en las tablas de documentación y encuestas, en cambio en las de autenticación y traslados no se presentaron modificaciones.

Entidades	***Primera versión***	***Versión corregida***	***Porque se corrigió***
documento	Tenía atributo codigo_qr propio	Se elimina codigo_qr	El código QR deja de generarse por cada documento individual.

categoría	Solo tenía id_categoria, nombre, descripcion	Se agregan codigo_qr (único, generado automáticamente al dar de alta la categoría) y tipo_categoria ('documento' / 'encuesta')	La categoría pasa a ser el punto de acceso físico del paciente: cada categoría tiene su propio QR, y el tipo determina si ese QR muestra un listado de documentos o abre un formulario de encuesta.
encuesta	Entidad independiente, sin FK a categoría; tenía el atributo es_anonima (booleano)	Se agrega id_categoria (FK, nullable); se elimina es_anonima	La encuesta debe poder resolverse desde el QR de una categoría de tipo 'encuesta'. El anonimato deja de ser una opción configurable por encuesta, por lo que el campo booleano deja de tener sentido.
respuesta_encuesta	Tenía ci_paciente como FK a paciente	Se elimina ci_paciente y su FK	La encuesta es siempre anónima: conservar la cédula, aunque fuera opcional, contradiría ese requisito y permitiría reconstruir quién respondió.
paciente	Participaba del Bloque 2, relacionado con respuesta_encuesta mediante la relación "completa"	Deja de participar del Bloque 2; solo se relaciona dentro del Bloque 3 (con solicitud_traslado)	Al eliminarse ci_paciente de respuesta_encuesta, paciente pierde todo vínculo con documentación y encuestas.

5.1.2 Justificación

usuario representa a cualquier persona que usa el sistema. Se eligió **id_usuario** como PK en lugar del nombre de usuario, porque este último podría cambiarse y no conviene usarlo como clave de relación. El atributo **contrasenha_hash** nunca guarda la contraseña en texto plano, y **activo** permite el borrado lógico sin romper referencias históricas.

rol se modela como entidad separada y no como atributo de **usuario**, porque un mismo usuario puede tener más de un rol a la vez. **usuario_rol** es la tabla puente que resuelve esa relación N:N, con clave primaria compuesta (**id_usuario**, **id_rol**).

categoría agrupa documentos y concentra el punto de acceso por código QR, cada categoría tiene su propio **codigo_qr**, generado al momento del alta. El atributo **tipo_categoria** ('**documento**' o '**encuesta**') determina si el QR muestra un listado de documentos o redirige a un formulario.

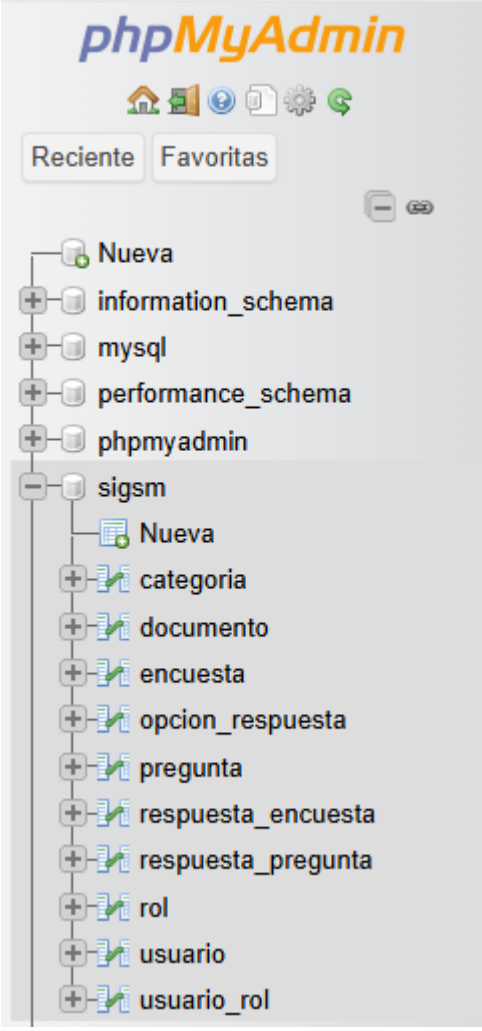
documento ya no tiene **codigo_qr** propio, porque el acceso ahora ocurre a nivel de categoría.

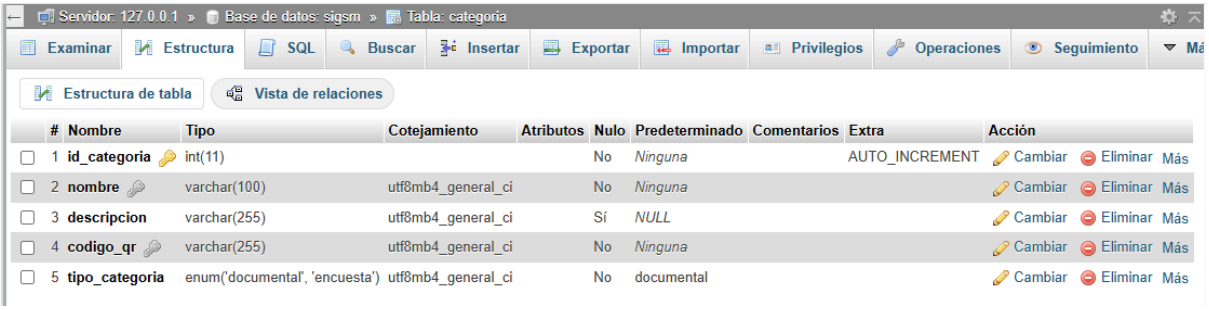
encuesta, **pregunta**, **opcion_respuesta**, **respuesta_encuesta** y **respuesta_pregunta** forman un patrón para guardar un número variable de preguntas sin cambiar 1FN. Las encuestas son siempre anónimas, ninguna de estas tablas guarda una referencia al paciente que respondió.

5.2 Implementación en PHPMyAdmin con scripts DDL

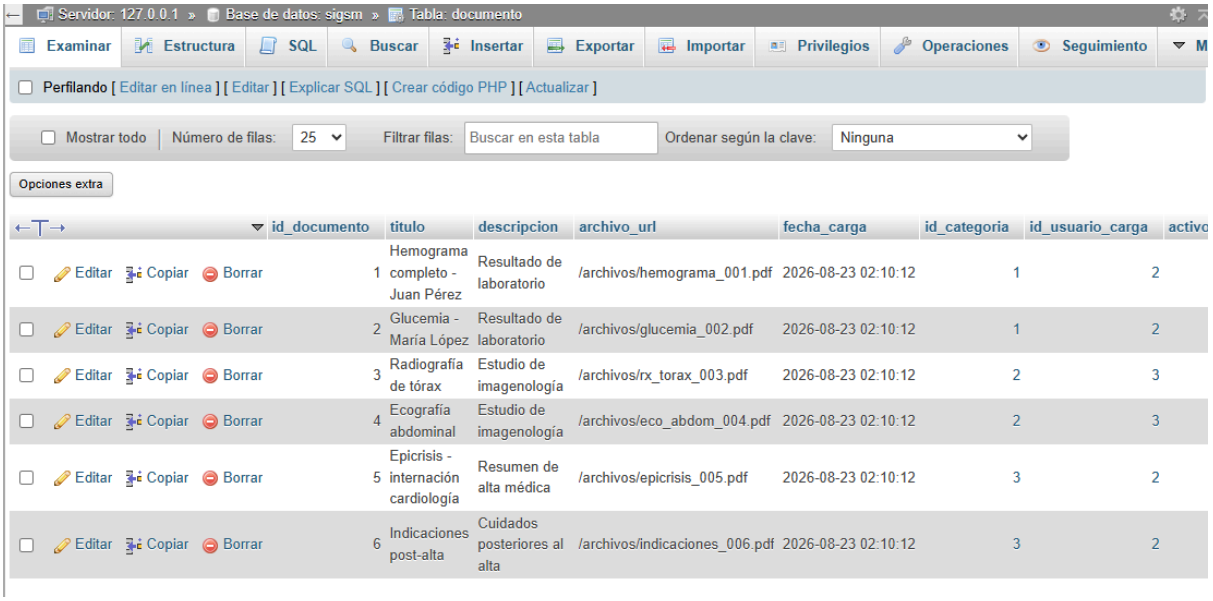
Se creó la base de datos con el nombre “sigsm” utilizando MySQL y PHPMyAdmin. En la misma se implementaron las tablas necesarias para el funcionamiento del sistema. La estructura de base de datos fue creada mediante sentencias SQL “CREATE TABLE”, en cada tabla se definieron las claves primarias (primary key) claves foráneas (foreign key) y restricciones necesarias para garantizar una correcta relación entre los datos.
Se adjunta en .sql la base de datos creada:
https://drive.google.com/file/d/1qw9tihqIzIFt07NfHxAjLxaRFClP7V5o/view?usp=drive_link

5.3 Evidencia de las tablas creadas









5.4 Datos de prueba

```
INSERT INTO `usuario` (`id_usuario`, `nombre_usuario`, `contrasenha_hash`, `nombre`, `apellido`, `email`, `activo`) VALUES
(1, 'eBanfi', '$2y$10$ZiB853b42WTDysHN1Xb4d.b0wbHUP0bwQPbvW1thdkVGnv0zMdDAK', 'Ezequiel', 'Banfi', 'eBanfi@isbo.edu.uy', 1),
(2, 'tLeites', '$2y$10$vqzdTW0Uan2Lr9MJ2aF/we0jsWvqGCCgzG5xrxmWogyjZwXk2oDVG', 'Thiago', 'Leites', 'tLeites@isbo.edu.uy', 1),
(3, 'mGomez', '$2y$10$mjl0XltDTpmTvUTxjUrnO.ZoBo4dbQiRgxItSAwHyi8cMIX0OVL6e', 'Mathias', 'Gómez', 'mGomez@isbo.edu.uy', 1),
(4, 'sAlmeida', '$2y$10$nV5ScDW1QQSY7wJch3izkOVj38JMa1YMScz3EG50UOTDUJj5zjEvC', 'Sabrina', 'Almeida', 'sAlmeida@isbo.edu.uy', 1);
```



5.5 Restricciones no estructurales

Restricciones	*Porque no se implementaron*
El código QR se genera automáticamente al crear la categoría.	Esto es lógica de aplicación, no algo que la base de datos pueda guardar o resolver sola.
Solo una categoría de tipo '**encuesta**' debe enlazar a **encuesta**.	Esto implicaría un **check** que consulte otra tabla.
Una encuesta es siempre anónima.	Ya está garantizado que **respuesta_encuesta** no tiene **ci_paciente**.
Un documento inactivo no debe listarse por QR.	Es un filtro de consulta (**WHERE activo = 1**).


N